Java Mini Project Report

Shapeville

Queen Mary University of London

Contributor:	Contribution:
Youhan Huang	Conducted integration testing, and wrote report.
Zhaobai Jiang	Designed main layout and progress tracking.
Zhixuan Guo	Built Bonus Tasks (Bonus 1 & 2)
Jiajun Li	Developed KS2 modules (Task 3 & 4)
Haoyu Liu	Implemented KS1 modules (Tasks 1 & 2)

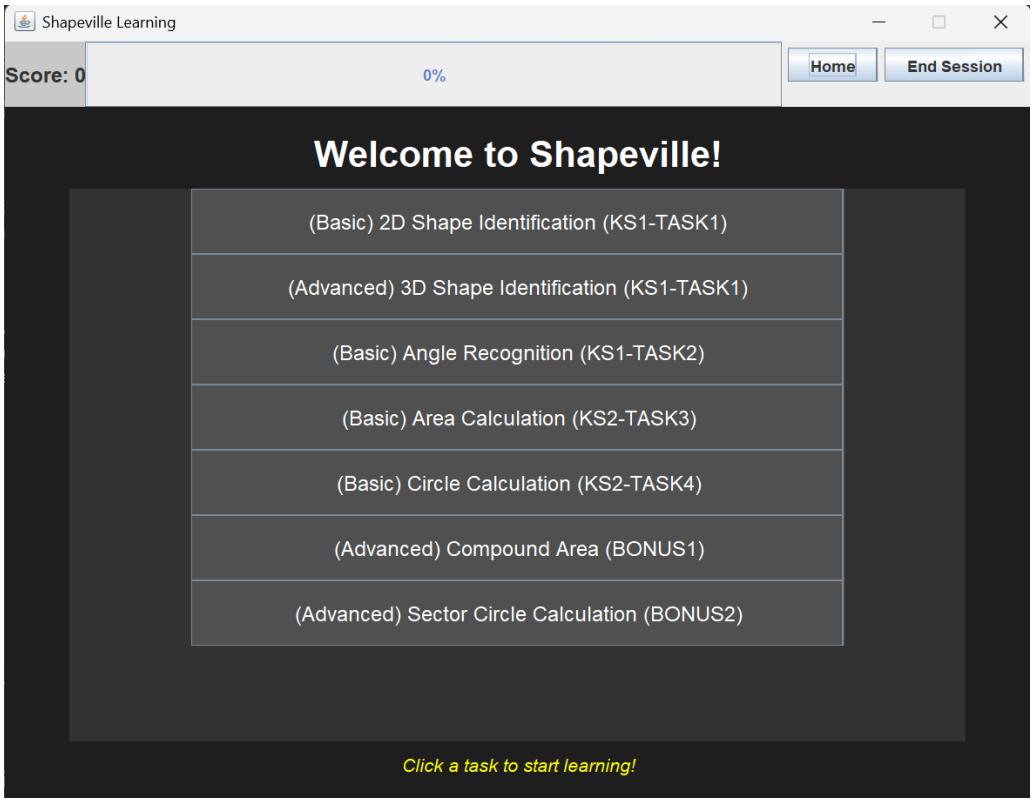

Figure 1: App Home Interface

Contents

1 System Design **3**
1.1 Flow Chart: Module Structure and Responsibilities . . . 3
1.2 Class diagrams . . . 4

2 Design Choices Justification **7**
2.1 Child-Friendly Design . . . 7
2.2 Inclusivity and Accessibility . . . 7
2.3 Engagement and Motivation Enhancements . . . 7
2.4 Modular and Maintainable Architecture . . . 7
2.5 Team-Friendly Codebase . . . 7

3 User Manual **8**
3.1 Running the Application from Command Line . . . 8
3.2 Launch & Home Screen . . . 8
3.3 Task Navigation . . . 8
3.4 How to Use Each Task . . . 8
3.5 Feedback and Scoring . . . 8

4 Tool Usage **9**
4.1 ChatGPT (GPT-4) . . . 9
4.2 GitHub Copilot . . . 9
4.3 Git & Branch-Based Collaboration . . . 9

5 Group Reflection **10**

6 Appendix: AI Prompts and Responses **11**
6.1 **Prompt 1** . . . 11
6.2 **Prompt 2** . . . 11
6.3 **Prompt 3** . . . 12

1 System Design

1.1 Flow Chart: Module Structure and Responsibilities

The system architecture is organized into clearly separated modules that reflect functional and visual responsibilities, as shown in Figure 3. Below is a textual overview of each major component:

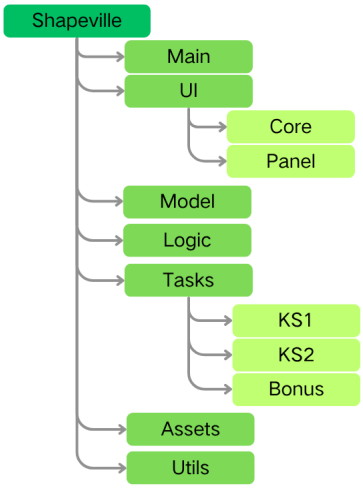

Figure 2: Program Architecture Overview

Main Module (`main`)

- `MainApp.java`: Entry point of the application. Initializes the main window and invokes setup.
- `MainFrame.java`: The top-level GUI container that switches between panels and renders the navigation bar.

Logic Module (`logic`)

- `TaskManager.java`: Central controller managing task transitions and user progression.
- `ScoreManager.java`: Computes and stores scores based on attempts and task types.
- `TaskLogic.java`: Contains task-specific logic abstractions used across multiple panels.

Model Module (`model.task`) Organized by task category to simplify reuse and debugging.

- **Bonus Tasks** (`bonus`): `Compound.java`, `Sector.java`
- **KS1 Tasks** (`sk1`):
  - `Task1Panel2D.java`: Panel for 2D shape identification.
  - `Task1Panel3D.java`: Panel for 3D shape identification.
  - `Task2Panel.java`: Panel for angle type recognition.
- **KS2 Tasks** (`sk2`):
  - `Task3Panel.java`: Panel for area calculation of standard shapes.
  - `Task4Panel.java`: Panel for circle area and circumference.

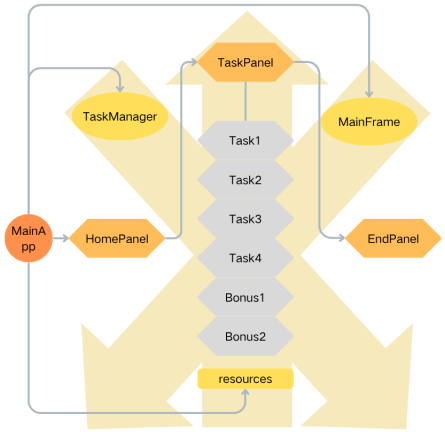

Figure 3: Data Flow

UI Template Module (`ui.panel_templates`)

- `HomeScreenPanel.java`: Displays available tasks, session progress, and navigation options.
- `TaskPanel.java`: Abstract class extended by all task-specific panels for consistent layout.
- `EndPanel.java`: Displayed after "End Session", showing summary of user progress.
- `NavigationBar.java`: Contains "Home" and "End Session" buttons; appears on all screens.

Utilities Module (`utils`)

- `Constants.java`: Stores global values (e.g., scoring constants, messages).
- `ImageLoader.java`: Loads task-related images from the resources folder.

Assets Directory (`resources/assets`)

- `2d`, `3d`: Images for Task 1.
- `compound`: Diagrams for Bonus 1.
- `sectors`: Diagrams for Bonus 2.

1.2 Class diagrams

The following tables describe the structure of key classes, their variables, and methods, to reflect the system's modular and task-driven architecture.

Class: `MainFrame`

Variables	`Map<String, JPanel> registeredPanels`
	`TaskManager taskManager`
	`ScoreManager scoreManager`

Methods	`registerPanel(String, JPanel)`
	`registerCorePanels()`
	`navigateToHome()`

Class: `TaskManager`

Variables	`List<TaskDefinition> masterTaskList` `List<String> sessionTaskSequenceIds`

Methods	`TaskManager(MainFrame, ScoreManager)`
	`defineMasterTasks()`
	`defineDefaultSessionSequence()`
	`setTaskSequence()`
	`resetCurrentTask()`

Class: `Task1Panel2D`

Variables	`MainFrame mainFrame`
	`JLabel shapeImageLabel`
	`JTextField answerTextField`
	`JButton submitButton`
	`JLabel resultLabel, scoreLabel, attemptsLabel`

Methods	`initializeUI()`
	`resetState()`
	`checkAnswer()`

Class: `Task2Panel`

Variables	`MainFrame mainFrame`
	`JLabel angleVisualLabel, feedbackLabel, attemptsLabel`
	`JTextField angleInputField`
	`JButton submitButton`
	`int currentAttempts, completedAngles`

Methods	`initializeUI()`
	`handleSubmit(ActionEvent)`
	`determineAngleType(int): String`
	`showCompletionDialog()`

Class: `Task3Panel`

Variables	`JLabel questionLabel, resultMessageLabel, formulaLabel`
	`JTextField answerField`
	`JPanel shapeDrawingPanel`
	`JButton submitButton`

Methods	`handleSubmit(ActionEvent)` `updateShapeButtons()`

Class: `Task4Panel`

Variables	`JLabel questionLabel, resultMessageLabel, formulaLabel`
	`JTextField answerField`
	`JPanel circleDrawingPanel`
	`JButton submitButton`

Methods	`startNewCircleProblem(boolean)` `handleSubmit()`

Class: `Compound`

Variables	`List<CompoundShape> shapes`
	`JLabel shapeImageLabel, shapeDescriptionLabel`
	`int currentShapeIndex`

Methods	`setupUI()`
	`showCurrentShape()`
	`checkAnswer(String)`
	`recordScore(int)`

Class: `Shape`

Variables	`String name, imagePath, type`
Methods	`getType(): String` `toString(): String`

Enum: `AngleType`

Variables	`String displayName`
Methods	`getDisplayName(): String`

Utility Class: `Constants`

Variables (Static)	`String HOME_PANEL_ID, END_PANEL_ID`
	`String TASK_TYPE_SHAPE_IDENTIFICATION_2D`
	`int DEFAULT_MAX_ATTEMPTS, TASK_TIME_LIMIT_SEC`

Methods	(None – constants only)

2 Design Choices Justification

2.1 Child-Friendly Design

- Uses large icons, clean layouts, and minimal text to suit young learners aged 5–10.
- Each task is presented on its own screen, minimizing distractions and helping children focus.
- All visual questions are accompanied by intuitive diagrams and pictorial aids, making abstract concepts more concrete.
- Tasks include specific guidance such as *"Only enter the whole number part"* to help clarify input expectations.
- Once all sub-tasks in a module are completed, the application automatically exits to the summary screen—removing the need for additional navigation and reducing friction.
- Motivational messages such as **"Great job!"**, **"Keep it up!"**, and **"You're doing well!"** are displayed throughout to build confidence and sustain attention.

2.2 Inclusivity and Accessibility

- A black-and-white dominant color scheme was chosen to maximize contrast and support learners with color vision deficiencies.
- No critical information is conveyed through color alone; all shapes and UI elements include text labels or visual differences in shape, line, or size.
- Instructions and hints are phrased in simple, action-oriented language, with optional visual or text-based repetition.
- Children can switch tasks at any time using the consistent navigation bar, ensuring flexibility and avoiding frustration for those with limited attention spans.

2.3 Engagement and Motivation Enhancements

- Time-limited tasks in the Bonus sections (5 minutes each) introduce a healthy sense of urgency and challenge, which increases engagement.
- Immediate visual feedback is given for correct or incorrect answers, and formulas are shown after three failed attempts—reinforcing learning through repetition.
- By progressively unlocking more advanced tasks, students experience a sense of progression and achievement.

2.4 Modular and Maintainable Architecture

- Each task is built on a shared `TaskPanel` framework, ensuring UI consistency while allowing independent development.
- Score calculation, progress tracking, and logic handling are centralized within the `logic` module, making debugging and feature extension easier.
- The codebase is organized by educational stages (KS1, KS2, Bonus), matching both the pedagogy and the team's workflow.

2.5 Team-Friendly Codebase

- The project structure enables parallel work by different team members across modules without conflict.
- Use of Git branching kept development clean and modular; integration was done only after local testing.
- Asset folders (e.g., `2d`, `compound`) are logically organized and reused across panels, reducing duplication and easing updates.

3 User Manual

3.1 Running the Application from Command Line

To compile and launch the application via command line, follow the instructions below:

Step 1: Compile the Project Ensure you are in the root directory of the project. Then run the following command to compile all Java files and output the ".class" files into the "bin" folder:

```
javac -d "bin" -sourcepath "src/main/java" -ncoding UTF-8 @"sources.txt"
```

Step 2: Run the Application After successful compilation, launch the application by running:

```
java -cp "bin" com.shapeville.main.MainApp
```

Note: - Make sure the "sources.txt" file contains the list of source ".java" files. - The working directory should contain both the "src" and "bin" folders.

3.2 Launch & Home Screen

Upon opening, the application shows a **Home Screen** with:

- List of available tasks (KS1, KS2, Bonus).
- A **Progress Bar** showing session progress.
- Navigation buttons: [**Home**] and [**End Session**].

3.3 Task Navigation

- Click on any task to open its dedicated panel.
- At any time, click **Home** to return or **End Session** to quit.

3.4 How to Use Each Task

KS1: Shape & Angle Tasks

- For **Shape Identification**, identify the name of shown 2D/3D shapes.
- For **Angle Types**, enter a degree value (10–350) and identify the angle type.

KS2: Area & Circle Tasks

- For **Area Tasks**, the system shows random dimensions. Use the formula to compute area within 3 minutes.
- For **Circle Tasks**, select Area or Circumference, then compute based on radius or diameter.

Bonus Tasks

- In **Compound Shapes**, compute area for given values of complex shapes.
- In **Sector Tasks**, calculate the area of a sector using provided radius and angle.

3.5 Feedback and Scoring

- Scores are based on correct attempt number (1st, 2nd, or 3rd).
- After each task, users receive feedback like **"Great job!"** and their score is updated.

4 Tool Usage

During the development of *Shapeville*, our team made use of several tools to enhance efficiency, improve code quality, and streamline collaboration:

4.1 ChatGPT (GPT-4)

We used ChatGPT primarily for:

- Exploring design ideas (e.g., task structure, scoring logic).
- Refining report writing (structure and phrasing).
- Asking clarification questions regarding UI accessibility and educational design best practices.

ChatGPT helped us ideate, but we manually adapted or rewrote its responses before using them in our implementation.

4.2 GitHub Copilot

Copilot was used during code implementation in VSCode:

- Suggested auto-completions for GUI and logic components.
- Helped reduce boilerplate coding (e.g., input validation, simple getters/setters).
- Occasionally suggested event listener patterns for Java Swing.

All Copilot-generated code was reviewed and tested by team members before being integrated.

4.3 Git & Branch-Based Collaboration

We adopted a **Git-based branch management strategy**:

- Each team member worked on a dedicated feature branch.
- All branches were merged into the `main` branch only after passing local tests and peer review.
- Conflicts were resolved during merge requests through regular group meetings.

This strategy helped isolate development tasks, reduce merge conflicts, and ensure smooth integration.

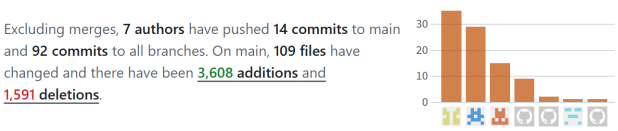

Figure 4: GitHub Using Record 1

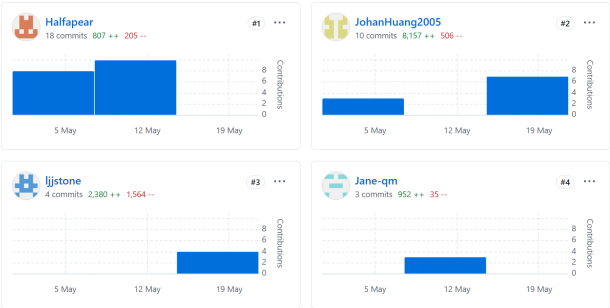

Figure 5: GitHub Using Record 2

5 Group Reflection

This mini-project was a collaborative and eye-opening experience for our team. The modular design structure—clearly dividing KS1, KS2, and Bonus tasks—proved effective in distributing responsibilities. One of our major successes was the seamless integration of individually developed modules and the creation of a coherent, child-friendly user interface.

Throughout the process, we also gained hands-on experience with essential software engineering practices:

- For most of us, this was our first time using **Git for team collaboration**. We learned to manage branches, handle commits and merges, and resolve conflicts. This greatly enhanced our understanding of real-world development workflows.
- Every member significantly improved their **Java development skills** through practical coding. In particular:
  - **Zhaobai** demonstrated strong architectural thinking and UI development skills.
  - **Youhan** excelled in debugging and team coordination.
  - **Jiajun**, **Zhixuan**, and **Haoyu** effectively implemented task-specific methods and integrated external contributions.

We also encountered some challenges:

- Git merge conflicts occasionally led to duplicated efforts when not carefully reviewed.
- Our program architecture lacked full maturity early on, which led to time pressure during final integration.
- Some of the initial implementations of tasks and bonus modules did not fully align with the original specifications and required later adjustments.

In retrospect, we would start UI design earlier, keep architectural planning more up-to-date, and conduct informal user testing (e.g., with children or parents) to refine the learning experience.

Overall, we gained a great deal from this project—not only in technical capability but also in collaborative development—and we will apply these lessons in our future work.

6 Appendix: AI Prompts and Responses

Below are the three key prompts submitted to AI tools, along with a note on how we used (or modified) each response.

6.1 Prompt 1

Prompt:

"Help me define Java classes for a modular shape-learning app for children, following MVC design."

Clarification:

MVC (Model-View-Controller) is a common software design pattern that separates application logic into:

- **Model**: Represents the data (e.g., shape objects).
- **View**: Responsible for the UI rendering.
- **Controller**: Handles interactions and logic flow.

Response (excerpt):

Shape MVC Example

```
\begin{lstlisting}[language=Java]
// Model
public class Shape {
   private String name;
   private BufferedImage image;
   public String getName() { return name; }
   ...
}

// View
public class ShapePanel extends JPanel {
   public void displayShape(Shape shape) {
       ...
   }
}

// Controller
public class TaskController {
   private Shape currentShape;
   public void handleAnswer(String userInput) {
       ...
   }
}
\end{lstlisting}
```

Figure 6: Prompt 1 Output

Usage Note:

We **did not directly use** this code. Instead, it inspired our own module separation between `model`, `ui`, and `logic` packages, but we designed our own architecture and class names.

6.2 Prompt 2

Prompt:

"Suggest ways to make a Java Swing GUI application accessible to color-blind children."

Response Highlights:

- Avoid red-green combinations.
- Use shape labels and visual icons in addition to color.

- Offer high-contrast mode or themes.
- Use tooltips and text descriptions.

Usage Note:
We implemented these ideas almost **as-is** in our GUI component designs. It directly influenced our decision to use textures, high-contrast visuals, and consistent labeling.

6.3 Prompt 3

Prompt:
"What would be a good scoring system for a children's educational app with limited retries?"
Response Highlights:

- Scoring by attempt (e.g., 3/2/1 points for 1st/2nd/3rd attempt).
- Show motivational feedback ("Well done!", "Keep trying!").
- Avoid harsh penalties to keep the user motivated.

Usage Note:
This approach was **adopted directly** and implemented in our `ScoreManager` class, integrated across all tasks.